Full RISC-V System Integration

ELEC 5803

Jesse Levine (101185127)

1 Final Accelerator Design

The Softmax accelerator is implemented as a high-performance hardware core designed to compute the normalized exponential function $\sigma(\mathbf{z})_i = \frac{e^{z_i - z_{max}}}{\sum e^{z_j - z_{max}}}$ using a vectorized, three-pass fixed-point architecture. By leveraging the Log-Sum-Exp trick (subtracting z_{max}), the design ensures numerical stability by keeping all exponent inputs ≤ 0, thus bounding results to the range $(0, 1]$.

1.1 Architecture

The architecture utilizes a task-level pipeline enabled by HLS DATAFLOW, allowing the hardware to overlap memory access with computation through hls::stream interconnections.

The execution begins with a vectorized load and max-search phase. The core reads input logits from the AXI interface in four-word bursts, utilizing a SIMD width of four to maximize bandwidth. As the data is streamed in, the hardware simultaneously populates a local BRAM buffer and performs a parallel reduction to identify the maximum logit in the vector. To support this level of parallelism, the internal BRAM buffers are cyclically partitioned. This memory interleaving ensures that the SIMD execution units can access four independent data elements in a single clock cycle without encountering bank conflicts or port contention.

Once the maximum value is determined, the accelerator proceeds to the exponential generation and summation phase. The core computes $e^{z_i - z_{max}}$ using a custom fixed-point approximation instead of an area-intensive Taylor series. This is achieved through piecewise linear interpolation; the hardware decomposes the input into an integer shift and a fractional lookup using a 17-entry table optimized for the Q16.16 format. These exponential values are then summed in a parallel accumulator tree. To finalize the operation without the latency of a standard hardware divider, the sum is passed through a piecewise linear reciprocal unit. This unit identifies the leading-one bit to normalize the sum, applies a linear fit ($y = mx + b$)

using pre-computed coefficients, and rescales the result to a Q2.30 inverse. This reciprocal is then multiplied by the cached exponentials in a final pipelined pass to produce the normalized probabilities.

1.2 Performance

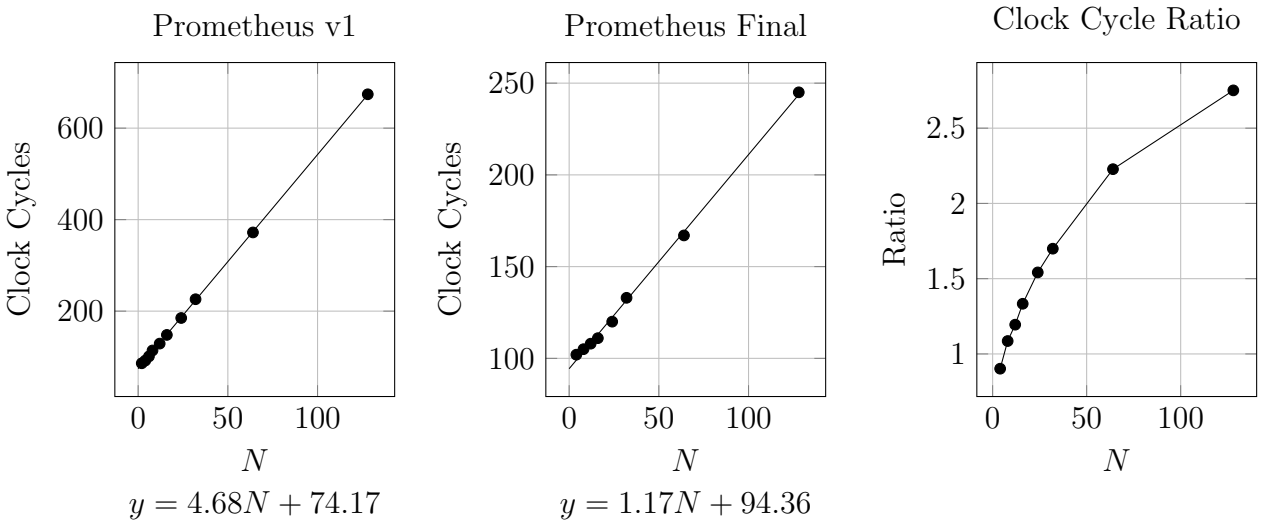

Figure 1: Clock cycle scaling comparison between the original Prometheus softmax implementation and the optimized architecture.

The performance of the final Softmax accelerator was characterized by sweeping the vector size (N) and measuring the resulting execution clock cycles. The empirical data reveals a linear scaling trend defined by the relationship $1.17N + 94.363$. At low values of N, the execution time is dominated by the constant overhead of approximately 94 cycles, which accounts for the initial AXI bus latency, the setup of the task-level dataflow pipeline, and the final reciprocal derivation. However, as N increases, the marginal cost per element remains extremely low, demonstrating the efficiency of the pipelined architecture and the low initiation interval of the internal loops.

To quantify the architectural improvements, the final version was benchmarked against the initial iteration, Prometheus_v1, which followed a trend line of $4.68N + 74.17$. While the initial version maintained a slightly lower fixed overhead, its processing slope was nearly four

times steeper than the final design. The comparative analysis of the clock cycle ratio between the two versions highlights a significant performance divergence as the workload grows. Specifically, for small vectors, the performance gain is modest; however, as N increases, the ratio of improvement scales aggressively, asymptotically approaching a 4x throughput boost.

This 4x performance ceiling aligns perfectly with the architectural transition from a scalar processing model to a 4-way SIMD (Single Instruction, Multiple Data) vectorized path. By interleaving BRAM banks and partitioning memory to feed four parallel processing lanes, the accelerator is able to perform four probability calculations per cycle. This result validates the effectiveness of the memory partitioning and the vectorized hardware-optimized arithmetic units, confirming that the bottleneck in the final design has been successfully shifted from computational throughput to the fundamental limits of the 4-word-per-cycle data path.

2 Core + Accelerator SoC Integration

This section details the system-level integration of the Softmax accelerator with the RISC-V processor core within the Prometheus SoC. The architecture employs a Memory-Mapped I/O (MMIO) strategy, allowing the CPU to treat the hardware accelerator as a set of standard memory addresses.

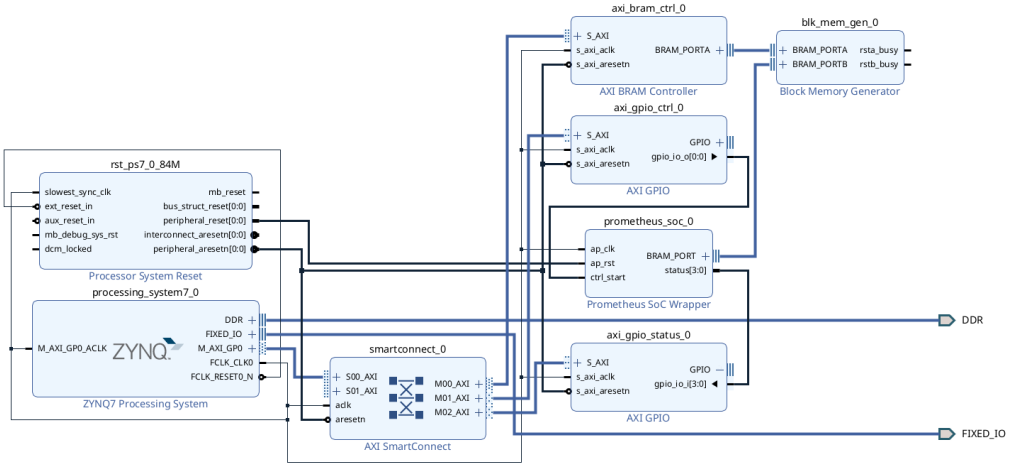

Figure 2: Vivado Block Diagram showing the integration of the Prometheus Softmax accelerator with the PYNQ-Z1 processing system via ACI SmartConnect and shared BRAM

2.1 System-Level Integration & MMIO Interface

The integration is centered around a unified memory map where the RISC-V core and the Softmax accelerator share access to a global BRAM via a dual-port configuration. The core of the integration logic resides in the accel_mmio_addr function and the corresponding dispatch logic within the CPU's load/store execution stages. Specific address ranges ($0x7000$ to $0x7014$) are reserved to act as the hardware-software interface. This range is mapped to virtual registers that control the accelerator's behavior: ACCEL_INPUT_BASE (input pointer), ACCEL_PROB_BASE (output pointer), ACCEL_N_REG (vector size), and ACCEL_AP_CTRL (command/status register).

When the RISC-V software executes a sw (store word) instruction to the control register (ACCEL_AP_CTRL) with the "Start" bit set, the CPU pauses its standard instruction flow to invoke the softmax_accel_core. Because both the CPU and the accelerator are bound to the same mem variable in HLS, they share the same physical BRAM resources. This "In-Memory" processing model is highly efficient; it eliminates the need for expensive DMA transfers or data copying between the CPU domain and the hardware domain. The CPU simply writes the physical addresses of the logit data into the MMIO registers, and the accelerator accesses that data directly.

By using this memory-mapped approach, the software driver is simplified to a few pointer-based assignments. The transition from software-only execution to hardware-accelerated execution is seamless: the CPU simply "configures and forgets," polling the status register until the hardware confirms that the probabilities have been computed and stored back in the shared memory space.

2.1.1 Functional Block Description

The physical realization of this architecture in the Vivado block design, Figure 2, consists of several critical components that facilitate the high-speed data path and control signaling between the ARM host and the custom logic. The ZYNQ7 Processing System serves as the

primary host controller and software-side foundation, providing the ARM cores, AXI master access into the Programmable Logic (PL), and the fundamental clock and reset sources. It is responsible for running the supervisory software that orchestrates memory initialization and monitors the execution of the hardware engine. Supporting this is the rst_ps7_0_84M reset controller, which synchronizes the asynchronous reset from the PS to the 84 MHz PL clock domain, ensuring a clean, glitch-free initialization for the AXI peripherals and the custom SoC block.

Communication and data routing are handled by the AXI SmartConnect and the AXI BRAM Controller. The SmartConnect acts as a sophisticated traffic router, arbitrating the single AXI master port from the PS to multiple memory-mapped destinations, including the control registers and the shared memory. The BRAM Controller specifically bridges the AXI protocol to the native memory interface, enabling the ARM PS to perform direct read and write operations on the Block Memory Generator (blk_mem_gen_0). This dual-ported BRAM serves as the central shared repository, storing the RISC-V program binary, the input logits, the final softmax probabilities, and critical debug values accessible by both the host and the accelerator.

The hardware-software handshake is managed by two dedicated AXI GPIO peripherals. The axi_gpio_ctrl_0 block acts as a write-only control register that the PS uses to assert the ctrl_start signal, effectively waking the hardware from an idle state. In the opposite direction, the axi_gpio_status_0 peripheral allows the PS to read real-time status bits—such as done, idle, and busy—emitted by the hardware. Finally, the Prometheus SoC Wrapper (prometheus_soc_0) represents the custom packaged IP containing the RISC-V core and the final Softmax accelerator. This block serves as the actual hardware engine, fetching instructions from the BRAM and executing the high-performance vectorized routines according to the parameters set by the PS.

3 Baseline Core vs SoC

This section evaluates the impact of the hardware accelerator on the overall system performance, moving beyond standalone hardware metrics to a full system-level comparison. By benchmarking the baseline RISC-V core—which performs the Softmax operation entirely in software—against the integrated SoC architecture, we can quantify the true efficiency gains of hardware-software co-design.

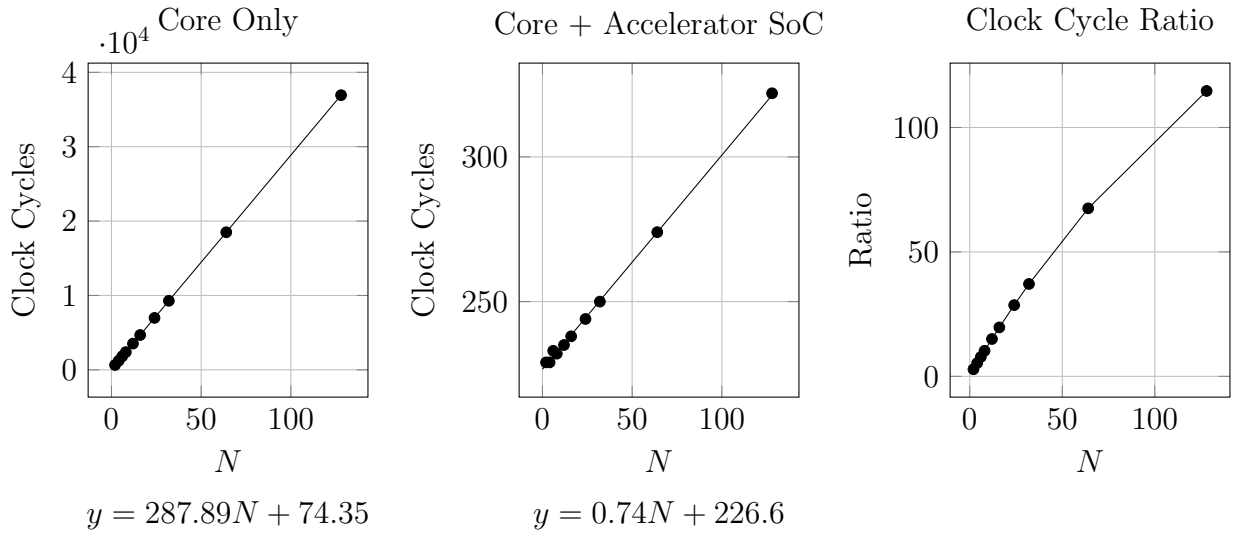

Figure 3: Clock cycle scaling comparison between the CPU-only implementation and the hardware-accelerated SoC implementation.

To evaluate the effectiveness of the integration, the system was characterized across a sweep of vector sizes (N), comparing the baseline execution on the RISC-V core alone against the unified Core + Accelerator SoC. The baseline core, executing a standard software-based Softmax routine, exhibited a steep linear scaling trend defined by $287.89N + 74.35$. In this configuration, the cycles required per element are significantly high due to the overhead of software loops, manual fixed-point normalization, and the lack of dedicated hardware for exponential approximations. Furthermore, as shown in Figure 3, the software implementation faced accuracy limitations at higher values of N, with the SoftMax sum deviating significantly from the ideal value of 1.0.

In contrast, the integrated Core + Accelerator SoC demonstrated a vastly superior scaling

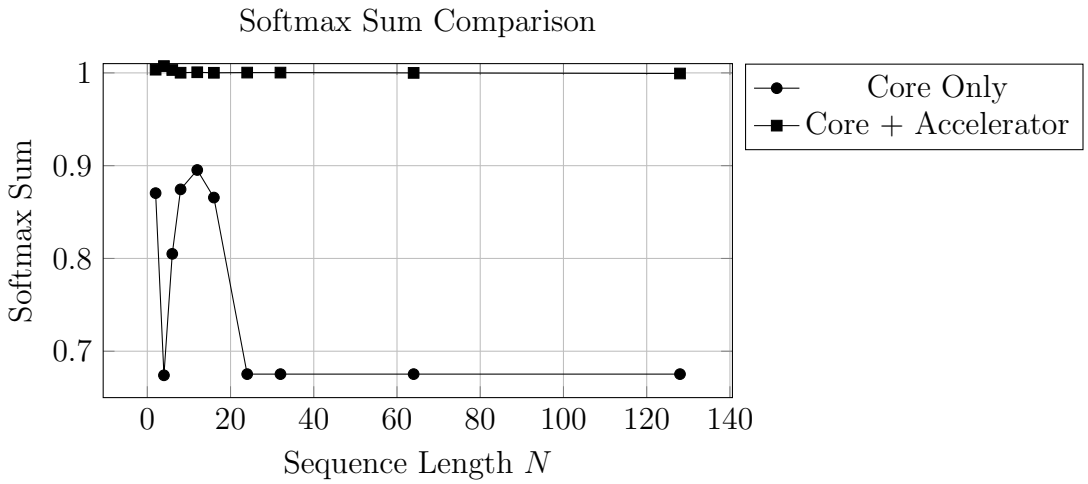

Figure 4: Softmax normalization accuracy for the CPU-only implementation and the accelerator-based SoC.

profile of $0.74N + 226.61$. While the SoC configuration incurs a higher fixed overhead of approximately 227 cycles—attributable to the MMIO register configuration and the hardware-software handshake—the marginal cost for adding an additional element is less than one clock cycle (0.74). This remarkable efficiency is a direct result of the 4-way SIMD vectorization and the pipelined dataflow, which allow the hardware to retire results much faster than the CPU can fetch and decode instructions. The SoC also restored numerical precision, maintaining a SoftMax sum near unity across the entire testing range.

The performance divergence becomes most apparent when analyzing the clock cycle ratio between the two systems. At the minimum vector size of $N = 2$, the SoC is already nearly 3x faster than the core alone. As N increases to 128, this speedup scales exponentially to over 114x. Mathematically, the performance boost asymptotes toward the ratio of the two slopes, specifically $\frac{287.89}{0.74} = 389$!!! This suggests that for large-scale workloads, the hardware-accelerated SoC provides a theoretical maximum throughput improvement of approximately 389x. This result confirms that offloading the computationally intensive Softmax task to specialized, vectorized logic effectively removes the CPU bottleneck, allowing the RISC-V core to focus on higher-level control flow while the hardware engine handles the bulk of the numerical processing.

4 AI Usage

I've now begun using Codex CLI for all my work. I love this era we are in.

References